Московский государственный университет имени М.В. Ломоносова

Направление 38.03.01 Экономика

Программа «Национальная экономика»

Прогнозирование рисков банковского кредитования с использованием технологий искусственного интеллекта

Выпускная квалификационная работа

Студент

Касенова Асем Ардаковна

Научный руководитель:

к.э.н., к.ф.-м.н., к.ю.н., доцент

Сидоренко Владимир Николаевич

Астана

2026

ОГЛАВЛЕНИЕ

Глава 1. Тестирование алгоритмов машинного и глубокого обучения на данных по кредитному скорингу . . . 3
1.1 Подбор гиперпараметров для алгоритмов машинного обучения при оценке возврата кредита . . . 3
1.1.1 Деревья решений . . . 4
1.1.2 Случайный лес . . . 7
1.1.3 Градиентный бустинг . . . 10
1.1.4 Нейронная сеть . . . 10
1.2 Ограничения и возможности моделей . . . 11

Приложение 1. Предобработка данных . . . 12

ГЛАВА 1. ТЕСТИРОВАНИЕ АЛГОРИТМОВ МАШИННОГО И ГЛУБОКОГО ОБУЧЕНИЯ НА ДАННЫХ ПО КРЕДИТНОМУ СКОРИНГУ

В предыдущей главе было выяснено, что алгоритмы градиентного бустинга и нейронных сетей не имеют эффектов переобучения и недообучения. В случае градиентного бустинга используются гиперпараметры по умолчанию, результаты метрик которых не ниже метрик других алгоритмов. Это означает, что не требуется поиск гиперпараметров для данного алгоритма и можно остановиться на гиперпараметрах по умолчанию. Идентичная ситуация возникает у нейронной сети с отличием в том, что у нее метрики точности немного хуже, чем у алгоритма градиентного бустинга. В этой главе будут подобраны гиперпараметры для алгоритмов деревьев решений и случайного леса, поскольку они обладают эффектом переобучения. Основным инструментом для поиска оптимальных гиперпараметров используются инструменты *GridSearchCV*[1] и *RandomizedSearchCV*[2]. Также в конце главы будут проанализированы прибыль и убыток банка на основании предсказаний четырех рассмотренных алгоритмов.

1.1 Подбор гиперпараметров для алгоритмов машинного обучения при оценке возврата кредита

Гиперпараметры – это такие параметры модели, которые задаются до начала работы модели. Правильно подобранные параметры позволяют улучшить качество предсказаний моделей. Каждому алгоритму МО присуще собственные наборы гиперпараметров, которые будут рассматриваться ниже. Существуют два метода подбора гиперпараметров:

1. *GridSearchCV* – метод подбора оптимальных гиперпараметров для модели с помощью перебора всех возможных комбинаций из заданного набора, что является ресурсозатратным способом.
2. *RandomizedSearchCV* – метод, позволяющий выбирать количество случайных комбинаций из заданного набора гиперпараметров. Является наиболее эффективным методом по времени при работе с большими данными.

[1] GridSearchCV [Электронный ресурс] / Python-библиотека для машинного обучения. URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html (дата обращения: 25.11.2025).

[2] RandomizedSearchCV [Электронный ресурс] / Python-библиотека для машинного обучения. URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html (дата обращения: 25.11.2025).

1.1.1 Деревья решений

Для алгоритма классификации решающих деревьев (*Decision Tree Classifier*) основными гиперпараметрами являются:

1. *criterion* – критерий качества разбиения, определяющая показатель, по которому алгоритм лучше классифицирует клиентов.
   a) *gini* – индекс Джини[3], измеряющий неоднородность двух классов (0 и 1) внутри узла. Если индекс равен 0, значит в узле содержатся объекты одного класса. Чем индекс ближе к 0, тем лучше алгоритм определяет класс клиента. Индекс Джини в алгоритме *Decision Tree* используется по умолчанию, поскольку быстрее вычисляется;
   b) *entropy* – энтропия Шеннона[4], измеряющий степень смешанности классов. Высокая энтропия свидетельствует о высокой неоднородности классов, чем ниже энтропия, тем узел более однородный. Алгоритм выбирает разбиения с наиболее низкой энтропией, т.е. содержатся большинство объектов одного класса;
   c) *log_loss* – логарифмическая функция потерь. В данном случае алгоритм не просто выбирает класс, а оценивает их вероятности. За каждый неверный прогноз вводит штраф.
2. *splitter* – параметр, задающий способ разбиения каждого узла дерева решений.
   a) *best* – алгоритм последовательно перебирает все возможные признаки и пороги, которые лучше всего разделяют клиентов по целевой переменной. При данном параметре требуется больше времени на обучение.
   b) *random* – в каждом узле дерева рассматривается случайная часть разбиения, среди которых выбирается лучшее значение. За счет этого модель быстрее обучается, но качество разбиений может быть немного хуже, чем при *best*.
3. *max_depth* – максимальная глубина дерева решений. Чем больше глубина, тем длиннее цепочка вопросов, чтобы выдать прогноз по клиенту. При небольших значениях алгоритм делает мало разбиений и недообучается, т.е. плохо объясняет данные и дает невысокую точность на обучающей и тестовой выборках. При высоких значениях алгоритм, наоборот, переобучается, запоминает все особенности обучающей выборки, но плохо классифицирует клиентов на тестовых данных.
4. *min_samples_split* – параметр задает минимальное число наблюдений в узле, при котором возможно следующее разбиение на два дочерних. Если в узле наблюдений меньше заданного порога, то дальнейшее разбиение не выполняется, и узел становится конечным.

[3]Джини К. (1884-1965 гг.) – итальянский статистик, экономист, социолог и демограф.
[4]Шеннон К.Э. (1916-2001 гг.) – американский инженер, криптоаналитик и математик, заложил основы теории информации.

5. $min_samples_leaf$ – параметр с минимальным заданным количеством наблюдений после разбиения в каждом узле, что позволяет дереву не создавать слишком маленькие группы клиентов и служит инструментов против переобучения алгоритма.
6. $max_features$ – параметр задает случайное количество признаков, среди которых алгоритм ищет оптимальное разделение.
   a) $sqrt$ – в каждой вершине выбирается количество признаков, равное квадратному корню из общего числа признаков.
   b) $log2$ – число признаков, равное двоичному логарифму от их общего количества.
7. $random_state = 42$ – произвольное число[5], задаваемое программе, чтобы при каждом запуске программы конечный результат оставался неизменным.

В таблице 1 указаны найденные гиперпараметры на 3 000 000 клиентов алгоритма деревья решений двумя методами $GridSearchCV$ и $RandomizedSearchCV$.

Таблица 1 – Выбор гиперпараметров для алгоритма $Decision\ Tree$ по двум методам

	GridSearchCV	RandomizedSearchCV
criterion	entropy	log_loss
splitter	best	best
max_depth	20	20
max_features	`sqrt`	`sqrt`
min_samples_leaf	20	15
min_samples_split	10	15
random_state	42	42

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

С помощью найденных гиперпараметров алгоритм решающих деревьев продемонстрировал следующие результаты (см. Таблицу 2, см. Таблицу 3). Из таблицы 2 видно, что после подбора гиперпараметров методом $GridSearchCV$ алгоритм дерева решений показал высокое качество классификации на обучающей и тестовой выборках. Высокие значения $precision$ указывают на то, что модель редко ошибочно определяет метку класса клиентов. Также значения $recall$ демонстрируют, что модель хорошо выявляет недефолтных и дефолтных заемщиков. Значения $f1 - score$ подтверждают сбалансированность между точностью и полнотой метрик. Метрика $accuracy$, равная 0.98, указывает на высокую долю верно классифицированных наблюдений, а $ROC - AUC$ на уровне 0.98–0.99 демонстрирует высокую способность модели разделять клиентов на два класса. Близость результатов метрик позволяет сделать вывод об отсутствии выраженного переобучения модели.

[5]Почему именно «42» - широко известное и запоминающееся значение по умолчанию среди исследователей. Это ответ на главный вопрос жизни, Вселенной и всего сущего в романе Дугласа Адамса «Автостопом по Галактике».

Таблица 2 – Метрики точности алгоритма Decision Tree с использованием *GridSearchCV*

Алгоритм – Деревья решений						
Flag метка	Precision точность	Recall полнота	f1–score f1–мера	Accuracy точность	ROC–AUC	Количество
Тренировочная выборка						
0	1.00	0.97	0.98	0.98	0.99	1438473
1	0.94	1.00	0.97			811527
Тестовая выборка						
0	1.00	0.97	0.98	0.98	0.98	478610
1	0.95	1.00	0.97			271390

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

Для более наглядной интерпретации полученных результатов представлена матрица ошибок (см. Рисунок 1). На обучающей выборке модель допустила 78 ошибок при классификации недефолтных клиентов и 44960 ошибок при классификации дефолтных клиентов. На тестовой выборке соответствующие значения составили 34 и 14912 ошибок. Снижение неверных предсказаний на тестовой выборке объясняется меньшим объемом этой выборки. Можно заметить, что структура ошибок на обучающей и тестовой выборках остается близкой, указывая на сохранение качества классификации на тестовых данных и отсутствии выраженного переобучения.

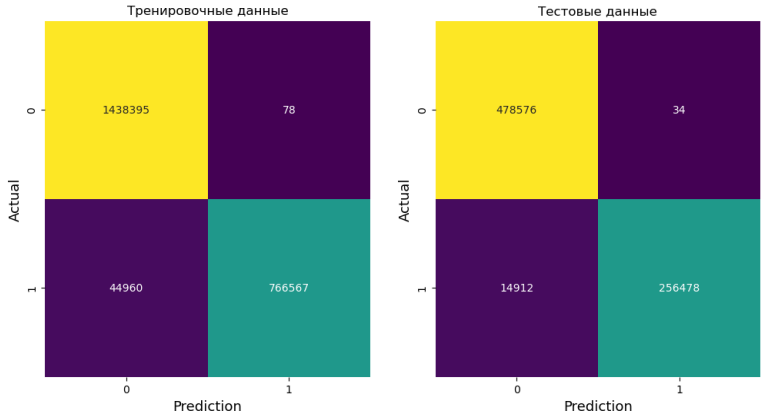

Рисунок 1 – Результаты подбора гиперпараметров с помощью *GridSearchCV*

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

Значения метрик, подобранные методом *RandomizedSearchCV* (см. Таблицу 2), остаются высокими и близкими к результатам, полученными методом *GridSearchCV*, поэтому основное внимание уделяется сравнению матриц ошибок.

Таблица 3 – Метрики точности алгоритма Decision Tree с использованием *RandomizedSearchCV*

Алгоритм – Деревья решений						
Flag метка	Precision точность	Recall полнота	f1–score f1–мера	Accuracy точность	ROC–AUC	Количество
Тренировочная выборка						
0	1.00	0.97	0.98	0.98	0.98	1438473
1	0.94	1.00	0.97			811527
Тестовая выборка						
0	1.00	0.97	0.98	0.98	0.98	478610
1	0.94	1.00	0.97			271390

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

Из рисунка 2 видно, что в данном случае модель допускает примерно в 12 раз ошибок как на обучающей, так и на тестовой выборках. Сравнив результаты ошибочных классификаций, выбор делается в пользу гиперпараметров, подобранных методом *GridSearchCV*.

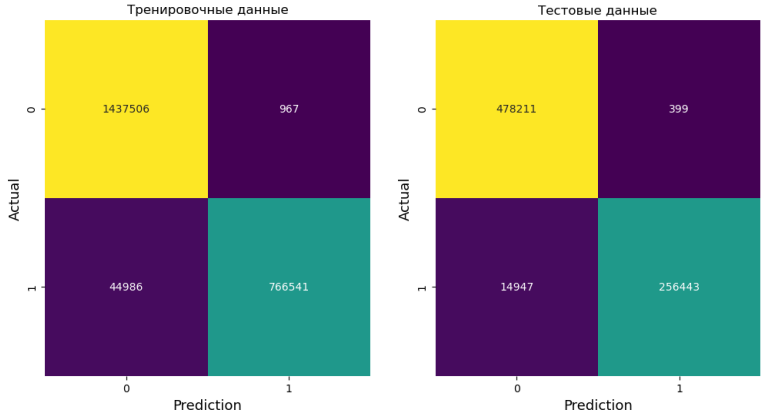

Рисунок 2 – Результаты подбора гиперпараметров с помощью *RandomizedSearchCV*

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

1.1.2 Случайный лес

Случайный лес является модификацией алгоритма деревья решений. По этой причине данные алгоритмы имеют схожие гиперпараметры, кроме $n_estimators$, где $n_estimators$ – это число деревьев в алгоритме случайного леса.

Таблица 4 – Выбор гиперпараметров для алгоритма *Random Forest* по двум методам

	GridSearchCV	RandomizedSearchCV
n_estimators	60	80
criterion	gini	log_loss
max_depth	20	20
max_features	`sqrt`	`log2`
min_samples_leaf	10	15
min_samples_split	30	20
random_state	42	42

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

Из таблицы 5 видно, что после подбора гиперпараметров с помощью $GridSearchCV$ алгоритм показал высокое качество классификации на обеих выборках. Высокие значения метрик $precision$, $recall$, $f1-score$ свидетельствуют о хорошем распозновании недефолтных и дефолтных клиентов. Значения $accuracy$ и $ROC-AUC$ также показывают хорошую разделяющую способность модели. Вместе с тем для класса 1 на обучающей выборке метрики немного выше, чем на тестовой, что указывает на небольшой эффект переобучения. Однако различия не являются существенными, поэтому модель сохраняет хорошую предсказательную способность.

Таблица 5 – Метрики точности с использованием $GridSearchCV$

Алгоритм – Случайный лес						
Flag метка	Precision точность	Recall полнота	f1–score f1–мера	Accuracy точность	ROC–AUC	Количество
Тренировочная выборка						
0	1.00	0.99	0.99	0.99	0.98	1438473
1	1.00	0.98	0.99			811527
Тестовая выборка						
0	1.00	0.97	0.98	0.98	0.98	478610
1	0.95	1.00	0.97			271390

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

Матрица ошибок (см. Рисунок 3) подтверждает результаты таблицы 5. Большинство наблюдений на обеих выборках верно классифицировано, а число ошибок остается примерно на прежнем уровне, что и у алгоритма $DecisionTree$. При этом относительная доля ошибок по обоим классам остается близкой, что указывает на хорошую устойчивость модели при классификации наблюдений.

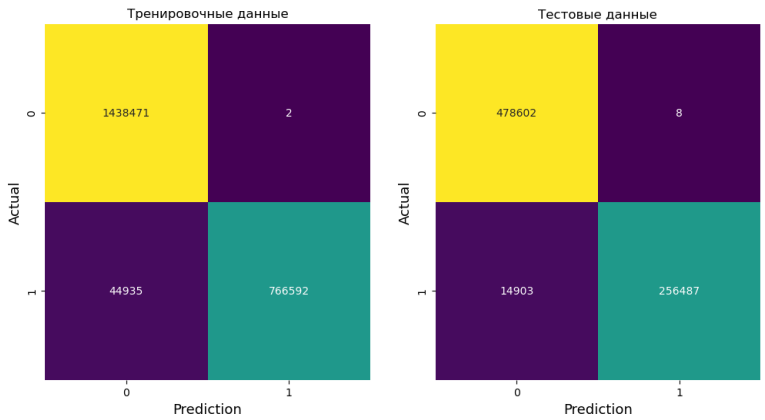

Рисунок 3 – Результаты подбора гиперпараметров с помощью *GridSearchCV*

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

Из таблицы 6 видно, что алгоритм с гиперпараметрами, подобранными методом *RandomizedSearchCV*, сохраняет высокое качество классификации на обеих выборках. Однако по отдельным метрикам для класса 1 результаты немного ниже по сравнению с *GridSearchCV*.

Таблица 6 – Метрики точности с использованием *RandomizedSearchCV*

Алгоритм – Случайный лес						
Flag метка	Precision точность	Recall полнота	f1–score f1–мера	Accuracy точность	ROC–AUC	Количество
Тренировочная выборка						
0	1.00	0.97	0.98	0.98	0.99	1438473
1	0.94	1.00	0.97			811527
Тестовая выборка						
0	1.00	0.97	0.98	0.98	0.98	478610
1	0.95	1.00	0.97			271390

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

Матрица ошибок (см. Рисунок 4) показывает, что модель с гиперпараметрами, подобранными методом *RandomizedSearchCV*, допускает больше ошибок по классу 0 по сравнению с вариантом *GridSearchCV*, при этом число ошибок по классу 1 немного ниже. Тем не менее общее число ошибок выше, чем при *GridSearchCV*, поэтому в данном случае целесообразно выбрать гиперпараметры, найденные с помощью *GridSearchCV*.

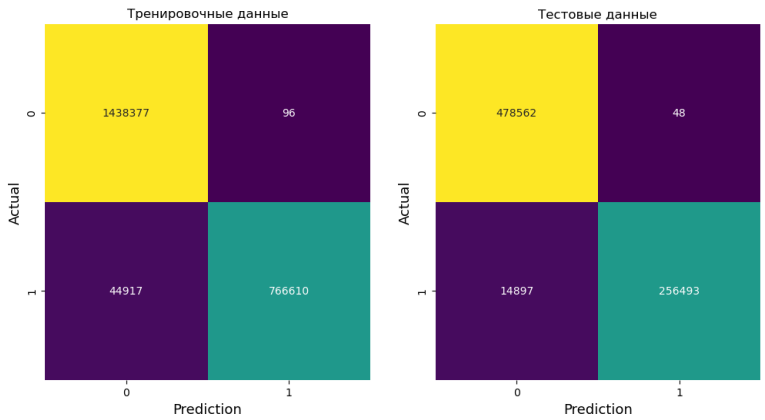

Рисунок 4 – Результаты подбора гиперпараметров с помощью *RandomizedSearchCV*

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

1.1.3 Градиентный бустинг

1.1.4 Нейронная сеть

Таблица 7 – Метрики точности Neural Network

Нейронная сеть						
Flag метка	Precision точность	Recall полнота	f1–score f1–мера	Accuracy точность	ROC–AUC	Количество
Тренировочная выборка						
0	1.00	0.97	0.98	0.98	0.97	1438473
1	0.94	1.00	0.97			811527
Тестовая выборка						
0	1.00	0.97	0.98	0.98	0.97	478610
1	0.95	1.00	0.97			271390

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

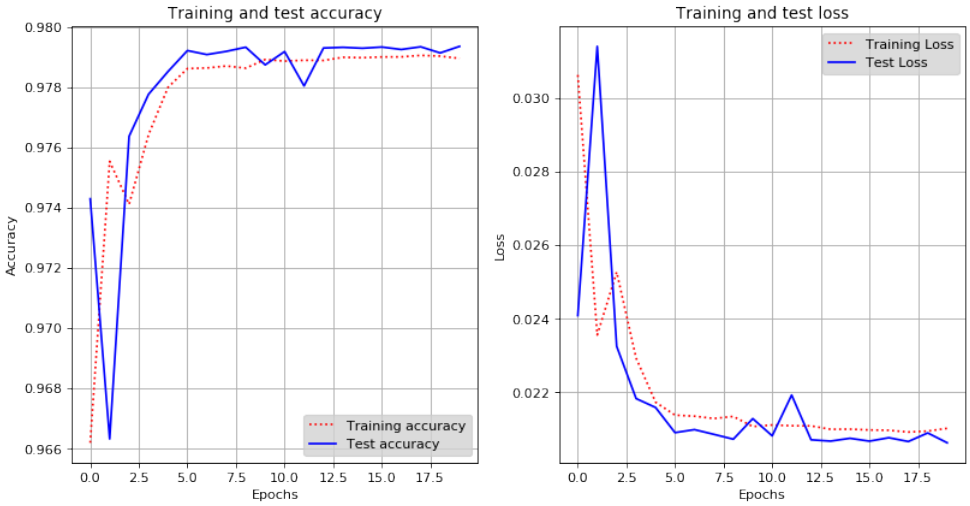

Рисунок 5 – Название придумать

Источник: составлено автором на основе: Соревнование на данных кредитных историй [Электронный ресурс] / Open Data Science. – URL: https://ods.ai/competitions/dl-fintech-bki (дата обращения: 25.11.2025).

1.2 Ограничения и возможности моделей

Приложение 1. Предобработка данных

Листинг 1 – Формирование единого датасета

```
import pandas as pd
import os
import pyarrow.parquet as pq

path = "train_data"
for i,file in enumerate(os.listdir(path)):
    file = 'train_data_' + str(i) + '.pq'
    dataset = pq.ParquetDataset(os.path.join(path,file))
    df = dataset.read(use_threads=True).to_pandas()
    df_gr = df.groupby('id').agg('mean')
    file_csv = file.replace('.pq', '.csv')
    df_gr.to_csv(os.path.join('train_data_csv_all',file_csv))
os.listdir(path)

path = 'train_data_csv_all'
frames = []
for file_csv in os.listdir(path):
    if file_csv.endswith('.csv'):
        df = pd.read_csv(os.path.join(path, file_csv))
        frames.append(df)

result = pd.concat(frames)
file_csv_all = '1_data_csv_all.csv'
result.to_csv(file_csv_all)
df_all = pd.read_csv(file_csv_all)
```

Листинг 2 – Датасет, содержащий 41 признак

```
from sklearn.decomposition import PCA

file_csv = 'data_csv_small.csv'
df = pd.read_csv(file_csv)

columns = ['pre_pterm', 'pre_fterm',
           'pre_loans_next_pay_summ','pre_loans_outstanding',
           'pre_loans_total_overdue', 'pre_loans_max_overdue_sum',
           'pre_loans_credit_cost_rate', 'is_zero_loans5',
           'is_zero_loans530', 'is_zero_loans3060',
           'is_zero_loans6090', 'is_zero_loans90',
           'pre_util','pre_maxover2limit',
           'is_zero_util', 'is_zero_over2limit',
           'enc_paym_0', 'enc_paym_1', 'enc_paym_2',
           'enc_paym_3', 'enc_paym_4', 'enc_paym_5',
           'enc_paym_6', 'enc_paym_7', 'enc_paym_8',
           'enc_paym_9', 'enc_paym_10', 'enc_paym_11',
           'enc_paym_12', 'enc_paym_13', 'enc_paym_14',
           'enc_paym_15', 'enc_paym_16', 'enc_paym_17',
           'enc_paym_18', 'enc_paym_19', 'enc_paym_20',
           'enc_paym_21', 'enc_paym_22',
           'enc_paym_23', 'enc_paym_24']

X = df.loc[:, columns].copy()
pca = PCA()
pca.fit(X)

100*pca.explained_variance_ratio_.round(3)
```

Листинг 3 – Датасет, содержащий 24 признака

```
file_csv = 'data_csv_small.csv'
df = pd.read_csv(file_csv)

columns_pay = ['enc_paym_0', 'enc_paym_1', 'enc_paym_2',
               'enc_paym_3', 'enc_paym_4', 'enc_paym_5',
               'enc_paym_6', 'enc_paym_7', 'enc_paym_8',
               'enc_paym_9', 'enc_paym_10', 'enc_paym_11',
               'enc_paym_12', 'enc_paym_13', 'enc_paym_14',
               'enc_paym_15', 'enc_paym_16', 'enc_paym_17',
               'enc_paym_18', 'enc_paym_19', 'enc_paym_20',
               'enc_paym_21', 'enc_paym_22', 'enc_paym_23',
               'enc_paym_24']

X_pay = df.loc[:, columns_pay].copy()
pca = PCA()
pca.fit(X_pay)

100*pca.explained_variance_ratio_.round(3)
```

Листинг 4 – Алгоритм выявления аномальных клиентов

```
df_y = pd.read_csv('train_target.csv')
y  = df_y.flag.values[:1000000].copy()
y_ = df_y.flag.values[:1000000].copy()

counter_norm, counter_anom = 0, 0
ind_norm, ind_anom = [], []
for i,x_pay in enumerate(np.array(X_pay)):
    if y_[i]==0 and min(x_pay)>0:
        counter_anom +=1
        ind_anom.append(i)
        y_[i]=1
    else:
        counter_norm +=1
        ind_norm.append(i)
```

Листинг 5 – Критерий Колмогорова-Смирнова

```
col = 'enc_paym_15'
T = X_pay.enc_paym_15
x00, x01 = T[(y==0) & (y_== 0)], T[ind_anom]
x11, x01 = T[(y==1)], T[ind_anom]
ind00 = (x00!=0) & (x00!=1) & (x00!=2) & (x00!=3) & (x00!=4) & (x00
    !=5)
ind01 = (x01!=0) & (x01!=1) & (x01!=2) & (x01!=3) & (x01!=4) & (x01
    !=5)
ind11 = (x11!=0) & (x11!=1) & (x11!=2) & (x11!=3) & (x11!=4) & (x11
    !=5)
x00, x11, x01 = x00[ind00], x11[ind11], x01[ind01]

def my_cdf(x1, x2, step):
    tm = max(max(x1), max(x2))
    F1,F2 = [], []
    n1, n2 = x1[x1>0].shape[0], x2[x2>0].shape[0]
    for t in np.arange(0, tm+0.1, step):
        F1.append(x1[(x1>0)&(x1<t)].shape[0])
        F2.append(x2[(x2>0)&(x2<t)].shape[0])
    F1, F2 = np.array(F1)/n1, np.array(F2)/n2
    K = max(abs(F1-F2))*np.sqrt( n1*n2/(n1+n2))
    return F1, F2, K, tm
step = 0.01
F01,  F11, K,  tm = my_cdf(x01, x11, step)
F01_, F00, K_, tm = my_cdf(x01, x00, step)
K, K_
```

Листинг 6 – Выявление сильно коррелирующих соседних признаков

```
all_c, nod_c, def_c, ano_c = [], [], [], []
month = []
for i in range(1, X_pay.shape[1]):
    all_c.append(np.corrcoef(np.array(X_pay).T[i-1], np.array(X_pay).
       T[i])[0,1])
    ind_nod = np.where(y_==0)[0]
    nod_c.append(np.corrcoef(np.array(X_pay).T[i-1][ind_nod], np.
       array(X_pay).T[i][ind_nod])[0,1])
    ind_def = np.where(y==1)[0]
    def_c.append( np.corrcoef(np.array(X_pay).T[i-1][ind_def], np.
       array(X_pay).T[i][ind_def])[0,1])
    ind_ano = ind_anom
    ano_c.append( np.corrcoef(np.array(X_pay).T[i-1][ind_ano], np.
       array(X_pay).T[i][ind_ano])[0,1])
    month.append(i)

mth = 0
n_all, n_nod, n_def, n_ano = X_pay.shape[0], len(ind_nod), len(
   ind_def), len(ind_ano)
R_all, r_nod, r_def, r_ano = all_c[mth], nod_c[mth], def_c[mth],
   ano_c[mth]
transform_f(r_nod, R_all, n_nod), transform_f(r_def, R_all, n_def),
   transform_f(r_ano, R_all, n_ano)
```

Приложение 2. Тестирование алгоритмов после предобработки данных

Листинг 7 – Алгоритм Decision Tree на 1 000 000 данных

```
from sklearn.tree import DecisionTreeClassifier

X_train, X_test, Y_train, Y_test = train_test_split(X_pay, y_,
   test_size=0.25, random_state=0)
dtree = DecisionTreeClassifier()
dtree.fit(X_train, Y_train)

y_pred_train = dtree.predict(X_train)
y_pred_test = dtree.predict(X_test)

print(classification_report(Y_train, y_pred_train))
print(classification_report(Y_test, y_pred_test))
```

Листинг 8 – Алгоритм Random Forest на 1 000 000 данных

```
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, Y_train, Y_test = train_test_split(X_pay, y_,
   test_size=0.25, random_state=0)
rfc = RandomForestClassifier()
rfc.fit(X_train, Y_train)

y_pred_train = rfc.predict(X_train)
y_pred_test = rfc.predict(X_test)

print(classification_report(Y_train, y_pred_train))
print(classification_report(Y_test,  y_pred_test))
```

Листинг 9 – Алгоритм Gradient Boosting на 1 000 000 данных

```
from sklearn.ensemble import GradientBoostingClassifier

X_train, X_test, Y_train, Y_test = train_test_split(X_pay, y_,
    test_size=0.25, random_state=0)
boost_gr = GradientBoostingClassifier()
boost_gr.fit(X_train, Y_train)

y_pred_train = boost_gr.predict(X_train)
y_pred_test = boost_gr.predict(X_test)

print(classification_report(Y_train, y_pred_train))
print(classification_report(Y_test,  y_pred_test))
```

Листинг 10 – Алгоритм Gradient Boosting на 1 000 000 данных

```
from tensorflow.keras.layers import Dense

y = y_
features = X_pay.shape[1]

X_train, X_test, Y_train, Y_test = train_test_split(X_pay, y_,
    test_size=0.25, random_state=0)

epochs = 200

model = keras.Sequential()
model.add(keras.Input(shape=(features,)))
model.add(Dense(32, activation='relu'))
model.add(keras.layers.Dense(1, activation='sigmoid'))

opt = keras.optimizers.Adam(learning_rate=0.005)
metrics = keras.metrics.BinaryAccuracy(name="accuracy", dtype=None,
    threshold=0.5)

model.compile(optimizer=opt, loss='mse', metrics=metrics)
fit_history = model.fit(X_train, Y_train,
                        batch_size=128,
                        validation_data=(X_test, Y_test),
                        epochs=epochs)
```

СПИСОК ЛИТЕРАТУРЫ

1. GridSearchCV [Электронный ресурс] / Python-библиотека для машинного обучения. — URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html. — Загл. с экрана.
2. RandomizedSearchCV [Электронный ресурс] / Python-библиотека для машинного обучения. — URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html. — Загл. с экрана.